

# Frontend System Design Essentials

## 7 Common Pitfalls

### 1

## Designing Only for the Happy Path

**Mistake:** Assuming network is fast, backend never fails, and users never make mistakes.

**Impact:** Works on your machine, breaks for real users.

**Better:** Treat failure as the default.



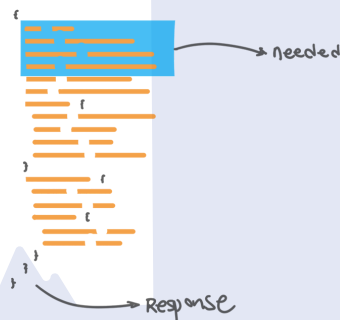
## Over-fetching or Under-fetching Data

### 2

**Mistake:** Fetching everything “just in case” or too little, causing repeated calls.

**Impact:** Sluggish UI, wasted bandwidth, or server overload.

**Better:** Fetch only what you need.



# Frontend System Design Essentials

## 7 Common Pitfalls

3

### No Request Management

**Mistake:** Letting in-flight requests pile up or overwrite each other.

**Impact:** Stale or incorrect data shown to users.

**Better:** Cancel stale requests, debounce inputs.



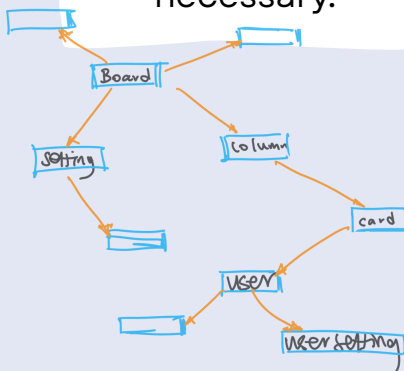
### Poor State Management

**Mistake:** Giant “god object” or scattered state everywhere.

**Impact:** Inconsistent UI, hard to debug/test.

**Better:** Normalize state, colocate local state, use global only when necessary.

4



# Frontend System Design Essentials

## 7 Common Pitfalls

### Lack of Error & Loading States

**Mistake:** Acting as if every request succeeds instantly.

**Impact:** Blank screens, confused users.

**Better:** Always handle loading, error, and empty states.

A diagram of a login form. It consists of two input fields: the first is labeled 'username@email.com' and the second is labeled 'password' with dots representing masked text. Below the password field is a blue button labeled 'Login'.

→ /api/login →

- downstream errors
- lost connection
- forgot password
- user doesn't exist
- password doesn't match
- database error
- JS error
- not accessible (screen reader)

5

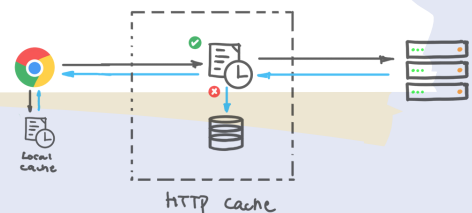
### No Caching Strategy

**Mistake:** Re-fetching the same data repeatedly.

**Impact:** Slow UI, wasted server resources.

**Better:** Cache smartly – in-memory, service workers, local storage.

6



# Frontend System Design Essentials

## 7 Common Pitfalls

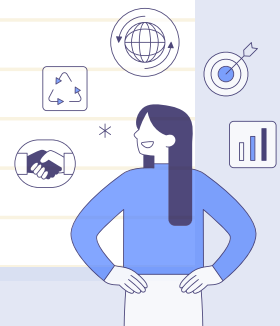
### Forgetting Accessibility & Internationalization

7

**Mistake:** Designing only for yourself (language, device, ability).

**Impact:** Excludes users, expensive fixes later.

**Better:** Accessibility + i18n from day one.



Want more practical frontend design tips?

Join my free newsletter The Pragmatic Developer →  
<https://juntao.substack.com>

Subscribe on YouTube for tutorials & live coding →  
<https://www.youtube.com/@icodeit.juntao>